

Blob Maintenance with Azure Functions

Student Lab: Timer Triggers, HTTP Triggers & Automated Blob Archiving

All commands use Windows Command Prompt (cmd.exe) | Fresh start — no prior lab required

DONE BY: Dharshini K

Lab Overview

In this lab you will build a serverless Blob maintenance service using Azure Functions. A Timer Trigger function will automatically move files older than 7 days from an uploads container to an archive container every night at midnight. An HTTP Trigger function gives you an on-demand way to run the same logic — perfect for testing. All Azure resources are created fresh from the command line.

What You Will Build

Lab	Goal
Lab 1: Setup	Sign in to Azure CLI, set variables, create the resource group
Lab 2: Storage	Create a storage account with uploads and archive containers
Lab 3: Key Vault	Store the storage connection string securely in Key Vault
Lab 4: Function App	Scaffold a .NET 8 isolated-worker Function App project
Lab 5: Timer Trigger	Write a function that archives old blobs on a nightly cron schedule
Lab 6: HTTP Trigger	Write an on-demand function that triggers the same archive logic
Lab 7: Test Locally	Run the Function App locally and test both functions with curl
Lab 8: Deploy to Azure	Create a Function App in Azure, connect Key Vault, and deploy
Lab 9: Cleanup	Delete all resources with a single command

Prerequisites — Install These First

- Azure CLI — confirm with: `az --version`
- .NET SDK 8.0 or later — confirm with: `dotnet --version`
- Azure Functions Core Tools v4 — install once with the command below, then confirm with: `func --version`

```
npm install -g azure-functions-core-tools@4 --unsafe-perm true
Meaning : global installation of azure functions core tools
```

Tip

If npm is not installed, download Node.js from <https://nodejs.org> — the LTS version includes npm. After installing Node.js, reopen cmd.exe and run the npm command above.

Important

Keep ONE cmd.exe window open for the entire lab. Variables set with the set command only last for the current session — if you close the window you must re-run the set commands from Lab 1 using your original SUFFIX value.

Lab 1: CLI Setup

Step 1 — Sign In

1. Run `az login`. A browser window will open — sign in with your Azure account.

```
az login
```

```
dhmarshinik@DHC47TYX72-dharshinik Ecommerce % az account show
{
  "environmentName": "AzureCloud",
  "homeTenantId": "2a731c61-a2b2-4661-8409-5b861cf40d0c",
  "id": "e25db867-f63d-489a-8bce-7de21e5f94a1",
  "isDefault": true,
  "managedByTenants": [],
  "name": "Training-2026",
  "state": "Enabled",
  "tenantDefaultDomain": "presidio.rocks.com",
  "tenantDisplayName": "Presidio Networked Solutions",
  "tenantId": "2a731c61-a2b2-4661-8409-5b861cf40d0c",
  "user": {
    "name": "dkarthik@presidio.com",
    "type": "user"
  }
}
```

I have already logged in

Step 2 — Set Variables

2. Type each line exactly. No spaces around the `=` sign.

```
set RG=func-lab-rg
set LOCATION=eastus
set /A SUFFIX=%RANDOM%
set SA_NAME=func-lab-sa-%SUFFIX%
set KV_NAME=func-lab-kv-%SUFFIX%
set FUNC_NAME=func-lab-app-%SUFFIX%
set CONTAINER_UPLOADS=uploads
set CONTAINER_ARCHIVE=archive
```

3. Confirm your values printed correctly:

```
echo %SA_NAME%
echo %KV_NAME%
echo %FUNC_NAME%
```

```

dharsinik@DHC47TYX72-dharsinik Ecommerce % export RG="rg_dharsini"
export LOCATION="southindia"
export SUFFIX=$RANDOM
export SA_NAME="dharsinifunclabsa${SUFFIX}"
export KV_NAME="dharsini-func-lab-kv-${SUFFIX}"
export FUNC_NAME="dharsini-func-lab-app-${SUFFIX}"
export CONTAINER_UPLOADS="dharsini-uploads"
export CONTAINER_ARCHIVE="dharsini-archive"
dharsinik@DHC47TYX72-dharsinik Ecommerce % echo $SA_NAME
dharsinifunclabsa29449
dharsinik@DHC47TYX72-dharsinik Ecommerce % echo $KV_NAME
dharsini-func-lab-kv-29449
dharsinik@DHC47TYX72-dharsinik Ecommerce % echo $FUNC_NAME
dharsini-func-lab-app-29449
dharsinik@DHC47TYX72-dharsinik Ecommerce % 

```

Write your values here — you will need them if you ever need to restore the variables:

SA_NAME value

dharsinifunclabsa29449

KV_NAME value

dharsini-kv-29449

FUNC_NAME value

dharsini-func-lab-app-29449

SUFFIX value

29449

Step 3 — Create the Resource Group

```
az group create --name %RG% --location %LOCATION%
```

```
and@SHINIK@DHC4711X72 and@SHINIK Ecommerce % az group show --name rg_dharshini
{
  "id": "/subscriptions/e25db867-f63d-489a-8bce-7de21e5f94a1/resourceGroups/rg_dharshini",
  "location": "southindia",
  "managedBy": null,
  "name": "rg_dharshini",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
```

Lab 2: Storage Account

The function will move blobs between two containers: uploads (where files land) and archive (where old files go). Both containers live in the same storage account.

Step 1 — Create the Storage Account

```
az storage account create ^
--name %SA_NAME% ^
--resource-group %RG% ^
--location %LOCATION% ^
--sku Standard_LRS ^
--kind StorageV2
```

```
dharshinik@DHC47TYX72-dharshinik Ecommerce % az storage account create \
--name $SA_NAME \
--resource-group $RG \
--location $LOCATION \
--sku Standard_LRS \
--kind StorageV2
{
  "accessTier": "Hot",
  "accountMigrationInProgress": null,
  "allowBlobPublicAccess": false,
  "allowCrossTenantReplication": false,
  "allowSharedKeyAccess": null,
  "allowSharedKeyAccessForServices": null,
  "allowedCopyScope": null,
  "azureFilesIdentityBasedAuthentication": null,
  "blobRestoreStatus": null,
  "creationTime": "2026-07-02T06:39:50.955841+00:00",
  "customDomain": null,
  "dataCollaborationPolicyProperties": null,
  "defaultToOAuthAuthentication": null,
  "dnsEndpointType": null,
  "dualStackEndpointPreference": null,
  "enableExtendedGroups": null,
  "enableHttpsTrafficOnly": true,
  "enableNfsV3": null,
```


[dharshinifunclabsa29449](#)


Storage account

StorageV2

[rg_dharshini](#)

South India

Step 2 — Create the Two Containers

```
az storage container create ^
```

```
--name %CONTAINER_UPLOADS% ^
--account-name %SA_NAME% ^
--auth-mode login

az storage container create ^
--name %CONTAINER_ARCHIVE% ^
--account-name %SA_NAME% ^
--auth-mode login
```

Read more about the command in [reference docs](#)

```
dharshtnik@DHC47TYX72-dharshtnik Ecommerce % az storage container create \
--name $CONTAINER_UPLOADS \
--account-name $SA_NAME \
--auth-mode login
{
  "created": true
}
```

```
dharshtnik@DHC47TYX72-dharshtnik Ecommerce % az storage container create \
> --name $CONTAINER_ARCHIVE \
> --account-name $SA_NAME \
> --auth-mode login
{
  "created": true
}
```

dharshtnifunclabsa29449 | Containers ☆ ...

Storage account

Search

+ Add container ↑ Upload ↻ Refresh | 🗑 Delete 🔒 Change access level ...

Search containers by prefix Only show active containers

Showing all 2 items

☐	Name	Last modified	Anonymous access level	Lease stat
☐	 dharshtni-archive	2/7/2026, 12:16:44 PM	Private	Available
☐	 dharshtni-uploads	2/7/2026, 12:15:43 PM	Private	Available

Overview
Activity log
Tags
Diagnose and solve problems
Access Control (IAM)
Data migration
Events

Step 3 — Capture the Connection String

(we are fetching the connection string from the storage account and storing it in a local environment variable)

```
export CONN_STRING=$(az storage account show-connection-string \
  --name $SA_NAME \
  --resource-group $RG \
  --query connectionString \
  -o tsv)
```

```
Echo $CONN_STRING
```

Tip

Verify it was captured: `echo %CONN_STRING%` — it should start with `DefaultEndpointsProtocol=https`

Lab 3: Key Vault

The connection string will be stored in Key Vault. In Azure, the Function App will read it via a Key Vault Reference — a special app setting syntax that tells Azure to fetch the value from Key Vault automatically, with no code changes needed.

Step 1 — Create the Vault

```
az keyvault create ^  
  --name %KV_NAME% ^  
  --resource-group %RG% ^  
  --location %LOCATION%
```

```

dhmarshinik@DHC47TYX72-dharshinik Ecommerce % az keyvault create \
--name $KV_NAME \
--resource-group $RG \
--location $LOCATION \
--enable-rbac-authorization false
{
  "id": "/subscriptions/e25db867-f63d-489a-8bce-7de21e5f94a1/resourceGroups
rg_dharshini/providers/Microsoft.KeyVault/vaults/dharshini-kv-29449",
  "location": "southindia",
  "name": "dharshini-kv-29449",
  "properties": {
    "accessPolicies": [
      {
        "applicationId": null,
        "objectId": "12900e06-4032-440a-8e35-46dd3958352a",
        "permissions": {
          "certificates": [
            "all"
          ],
          "keys": [
            "all"
          ],
          "secrets": [
            "all"
          ],
          "storage": [
            "all"
          ]
        },
        "tenantId": "2a731c61-a2b2-4661-8409-5b861cf40d0c"
      }
    ],
    "createMode": null,
    "enablePurgeProtection": null,
    "enableRbacAuthorization": false,
    "enableSoftDelete": true,
    "enabledForDeployment": false,

```

```
"vaultUri": "https://dharshini-kv-29449.vault.azure.net/"
```

Step 2 — Grant Yourself Access

4. Capture your Azure AD object ID and set an access policy:

```

for /f "delims=" %a in ('az ad signed-in-user show --query id -o tsv') do set MY_OID=%a

az keyvault set-policy ^
--name %KV_NAME% ^

```

```
--object-id %MY_OID% ^
--secret-permissions get set list delete
```

Step 3 — Store the Connection String

```
az keyvault secret set ^
--vault-name %KV_NAME% ^
--name "BlobStorageConnectionString" ^
--value "%CONN_STRING%"
```

```
dhmarshinik@DHC47TYX72-dharshinik Ecommerce % az keyvault secret set \
--vault-name $KV_NAME \
--name "BlobStorageConnectionString" \
--value $CONN_STRING
{
  "attributes": {
    "created": "2026-07-02T07:17:54+00:00",
    "enabled": true,
    "expires": null,
    "notBefore": null,
    "recoverableDays": 90,
    "recoveryLevel": "Recoverable+Purgeable",
    "updated": "2026-07-02T07:17:54+00:00"
  },
  "contentType": null,
  "id": "https://dharshini-kv-29449.vault.azure.net/secrets/BlobStorageConne
ctionString/b57113c710a947d99b44081396b7a84f",
  "kid": null,
  "managed": null,
  "name": "BlobStorageConnectionString",
  "tags": {
    "file-encoding": "utf-8"
  },
}
```

Connection String added to key vault like key value pairs

Important

This is the last time the connection string is handled directly. From this point the secret lives only inside Key Vault.

Lab 4: Scaffold the Function App

Step 1 — Create the Project

5. Choose a folder on your machine to work in, then `cd` into it.
6. Run the following to create the Function App project:

```
func init BlobArchiveFunctions --worker-runtime dotnet-isolated --target-framework net8.0
cd BlobArchiveFunctions
```

Tip

`func init` creates a `.csproj` file, a `Program.cs`, a `host.json`, and a `local.settings.json`. The `dotnet-isolated` model is the modern .NET approach for Azure Functions.

```
dh@SHINIK@DHC47TYX72-dharshinik Azure % cd Functions
dh@SHINIK@DHC47TYX72-dharshinik Functions % func init BlobArchiveFunctions
--worker-runtime dotnet-isolated --target-framework net10.0

Writing /Users/dharshinik/Desktop/Presidio/Genspark/Azure/Functions/BlobArch
iveFunctions/.vscode/extensions.json
dh@SHINIK@DHC47TYX72-dharshinik Functions % cd BlobArchiveFunctions
dh@SHINIK@DHC47TYX72-dharshinik BlobArchiveFunctions %
```

Step 2 — Add the Blob Storage Package

```
dotnet add package Azure.Storage.Blobs

//this package is used to interact with Azure Storage Blobs
```

```

dharshinik@DHC47TYX72-dharshinik BlobArchiveFunctions % dotnet add package Azure.Storage.Blobs
info : X.509 certificate chain validation will use the fallback certificate bundle at '/usr/local/share/dotnet/sdk/10.0.202/trustedroots/codesignctl.pem'.
info : X.509 certificate chain validation will use the fallback certificate bundle at '/usr/local/share/dotnet/sdk/10.0.202/trustedroots/timestampctl.pem'.
info : Adding PackageReference for package 'Azure.Storage.Blobs' into project '/Users/dharshinik/Desktop/Presidio/Genspark/Azure/Functions/BlobArchiveFunctions/BlobArchiveFunctions.csproj'.
info : GET https://api.nuget.org/v3/registration5-gz-semver2/azure.storage.blobs/index.json
info : OK https://api.nuget.org/v3/registration5-gz-semver2/azure.storage.blobs/index.json 410ms
info : Restoring packages for /Users/dharshinik/Desktop/Presidio/Genspark/Azure/Functions/BlobArchiveFunctions/BlobArchiveFunctions.csproj...
info : CACHE https://api.nuget.org/v3/vulnerabilities/index.json
info : CACHE https://api.nuget.org/v3-vulnerabilities/2026.07.01.00.04.29/vulnerability.base.json
info : CACHE https://api.nuget.org/v3-vulnerabilities/2026.07.01.00.04.29/2026.07.02.06.04.35/vulnerability.update.json
info : Package 'Azure.Storage.Blobs' is compatible with all the specified frameworks in project '/Users/dharshinik/Desktop/Presidio/Genspark/Azure/Functions/BlobArchiveFunctions/BlobArchiveFunctions.csproj'.
info : PackageReference for package 'Azure.Storage.Blobs' version '12.29.1' added to file '/Users/dharshinik/Desktop/Presidio/Genspark/Azure/Functions/BlobArchiveFunctions/BlobArchiveFunctions.csproj'.

```

Step 3 — Add the Timer Extension

```
dotnet add package Microsoft.Azure.Functions.Worker.Extensions.Timer
```

Step 4 — Create the Function Files

7. Add the Timer Trigger function file:

```
func new --name NightlyArchive --template "Timer trigger" --language C#
```

8. Add the HTTP Trigger function file:

```
func new --name ManualArchive --template "HTTP trigger" --language C#
```

```
dharshtnik@DHC47TYX72-dharshtnik BlobArchiveFunctions % func new \
  --name NightlyArchive \
  --template "Timer trigger"
Template: Timer trigger
Function name: [Timer trigger] NightlyArchive
The function "NightlyArchive" was created successfully from the "Timer trigger" template.
dharshtnik@DHC47TYX72-dharshtnik BlobArchiveFunctions % func new --name ManualArchive --template "HTTP trigger"
Template: HTTP trigger
Function name: [HTTP trigger] ManualArchive
The function "ManualArchive" was created successfully from the "HTTP trigger" template.
dharshtnik@DHC47TYX72-dharshtnik BlobArchiveFunctions %
```

Lab 5: Write the Timer Trigger Function

The Timer Trigger function runs automatically on a cron schedule. You will replace the generated placeholder code with logic that scans the uploads container and moves files older than 7 days to archive.

Step 1 — Create the Archive Service

9. Create a new file called `BlobArchiveService.cs` in the `BlobArchiveFunctions` folder with the following content:

```
using Azure.Storage.Blobs;
using Microsoft.Extensions.Logging;

namespace BlobArchiveFunctions;

public class BlobArchiveService
{
    private readonly ILogger<BlobArchiveService> _logger;
    private readonly string _connectionString;

    public BlobArchiveService(ILogger<BlobArchiveService> logger, IConfiguration config)
    {
        _logger = logger;
        _connectionString = config["BlobStorageConnectionString"]
            ?? throw new InvalidOperationException("BlobStorageConnectionString is not configured.");
    }

    public async Task<int> ArchiveOldBlobsAsync(int olderThanDays = 7)
    {
        var serviceClient = new BlobServiceClient(_connectionString);
        var uploads = serviceClient.GetBlobContainerClient("uploads");
        var archive = serviceClient.GetBlobContainerClient("archive");

        await archive.CreateIfNotExistsAsync();

        int count = 0;
        var cutoff = DateTimeOffset.UtcNow.AddDays(-olderThanDays);

        await foreach (var blobItem in uploads.GetBlobsAsync())
        {
            if (blobItem.Properties.LastModified < cutoff)
            {
                var sourceBlob = uploads.GetBlobClient(blobItem.Name);
                var destBlob = archive.GetBlobClient(blobItem.Name);

                await destBlob.StartCopyFromUriAsync(sourceBlob.Uri);
                await sourceBlob.DeleteAsync();

                _logger.LogInformation("Archived: {BlobName}", blobItem.Name);
                count++;
            }
        }
    }
}
```

```

        }
    }

    _logger.LogInformation("Archive run complete. Files moved: {Count}", count);
    return count;
}
}

```

Step 2 — Register the Service in Program.cs

10. Open Program.cs and replace its contents with:

```

using Microsoft.Azure.Functions.Worker;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using BlobArchiveFunctions;

var host = new HostBuilder()
    .ConfigureFunctionsWebApplication()
    .ConfigureServices(services =>
    {
        services.AddTransient<BlobArchiveService>();
    })
    .Build();

host.Run();

```

Step 3 — Replace the NightlyArchive Function

11. Open NightlyArchive.cs and replace its contents with:

```

using Microsoft.Azure.Functions.Worker;
using Microsoft.Extensions.Logging;

namespace BlobArchiveFunctions;

public class NightlyArchive
{
    private readonly BlobArchiveService _archiveService;
    private readonly ILogger<NightlyArchive> _logger;

    public NightlyArchive(BlobArchiveService archiveService, ILogger<NightlyArchive> logger)
    {
        _archiveService = archiveService;
        _logger = logger;
    }

    // Runs every day at midnight UTC
    // Cron format: {second} {minute} {hour} {day} {month} {day-of-week}

```

```
[Function("NightlyArchive")]
public async Task Run([TimerTrigger("0 0 0 * * *")] TimerInfo myTimer)
{
    _logger.LogInformation("NightlyArchive triggered at: {Time}", DateTime.UtcNow);
    await _archiveService.ArchiveOldBlobsAsync(olderThanDays: 7);
}
}
```

Tip

The cron expression 0 0 0 * * * means: at second 0, minute 0, hour 0, every day, every month, every day of the week — i.e. midnight UTC daily.

Lab 6: Write the HTTP Trigger Function

The HTTP Trigger lets you call the same archive logic on demand — useful for testing without waiting for midnight. It also accepts an optional query parameter so you can test with a shorter age window.

Step 1 — Replace the ManualArchive Function

12. Open ManualArchive.cs and replace its contents with:

```
using System.Net;
using Microsoft.Azure.Functions.Worker;
using Microsoft.Azure.Functions.Worker.Http;
using Microsoft.Extensions.Logging;

namespace BlobArchiveFunctions;

public class ManualArchive
{
    private readonly BlobArchiveService _archiveService;
    private readonly ILogger<ManualArchive> _logger;

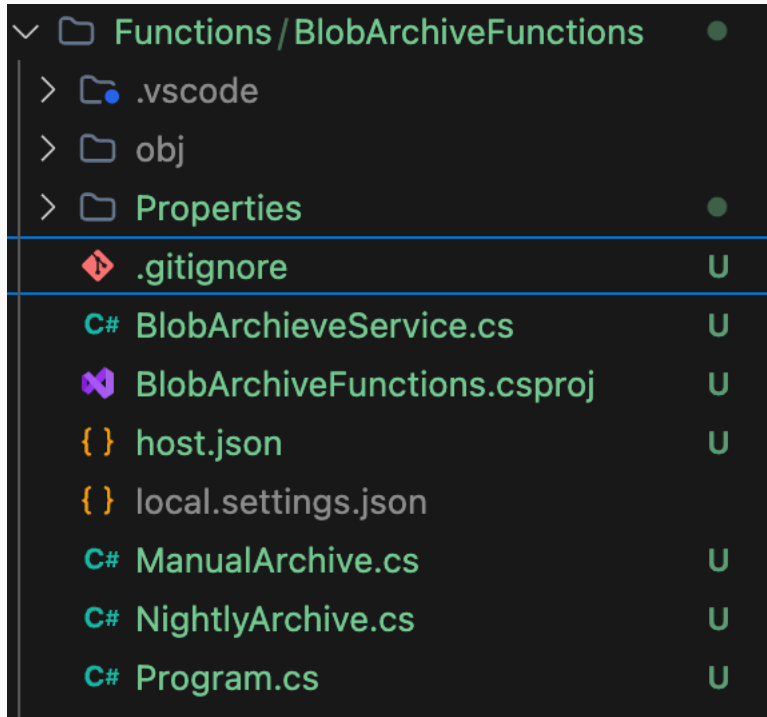
    public ManualArchive(BlobArchiveService archiveService, ILogger<ManualArchive> logger)
    {
        _archiveService = archiveService;
        _logger = logger;
    }

    [Function("ManualArchive")]
    public async Task<HttpResponseBody> Run(
        [HttpTrigger(AuthorizationLevel.Anonymous, "post")] HttpRequestData req)
    {
        // Optional: pass ?days=1 to archive files older than 1 day (useful for testing)
        var query = System.Web.HttpUtility.ParseQueryString(req.Url.Query);
        int days = int.TryParse(query["days"], out var d) ? d : 7;

        _logger.LogInformation("ManualArchive triggered – archiving files older than {Days} day(s).", days);

        int count = await _archiveService.ArchiveOldBlobsAsync(olderThanDays: days);

        var response = req.CreateResponse(HttpStatusCode.OK);
        await response.WriteStringAsync($"Archive complete. Files moved: {count}");
        return response;
    }
}
```



Lab 7: Test Locally

Before deploying to Azure, test the functions on your machine. The local settings file acts as a stand-in for Azure app settings when running locally.

Step 1 — Configure local.settings.json

13. Open local.settings.json (in the BlobArchiveFunctions folder) and replace its contents with your actual connection string:

```
{
  "IsEncrypted": false,
  "Values": {
    "AzureWebJobsStorage": "UseDevelopmentStorage=true",
    "FUNCTIONS_WORKER_RUNTIME": "dotnet-isolated",
    "BlobStorageConnectionString": "PASTE-YOUR-CONNECTION-STRING-HERE"
  }
}
```

//In AzureWebJobsStorage, set it to the connection string itself to connect directly to Azure Storage Account in Cloud. If UseDevelopmentStorage used, it will use Azurite to connect.

Important

Replace PASTE-YOUR-CONNECTION-STRING-HERE with the value from echo %CONN_STRING% in your cmd.exe session. local.settings.json is already in .gitignore by default — never commit it to source control.

Step 2 — Build the Project

```
dotnet build
```

```
dharshinik@DHC47TYX72-dharshinik BlobArchiveFunctions % dotnet build
Restore complete (0.4s)
  WorkerExtensions net8.0 succeeded (2.0s) → obj/Debug/net10.0/WorkerExtensions/bin/Release/net8.0/Microsoft.Azure.Functions.Worker.Extensions.dll
  BlobArchiveFunctions net10.0 succeeded (1.3s) → bin/Debug/net10.0/BlobArchiveFunctions.dll

Build succeeded in 67.4s
dharshinik@DHC47TYX72-dharshinik BlobArchiveFunctions %
```

Step 3 — Start the Function App

```
func start
```

The Functions runtime will start and print the URLs for both functions. Look for two lines like:

```
NightlyArchive: timerTrigger
ManualArchive: [POST] http://localhost:7071/api/ManualArchive
```

```
[2026-07-02T08:45:18.028Z] Found /Users/dharshinik/Desktop/Presidio/Genspark/Azure/Functions/BlobArchiveFunctions/BlobArchiveFunctions.csproj. Using for user secrets file configuration.
[2026-07-02T08:45:19.256Z] Worker process started and initialized.

Functions:

    ManualArchive: [POST] http://localhost:7071/api/ManualArchive

    NightlyArchive: timerTrigger

For detailed output, run func with --verbose flag.
[2026-07-02T08:45:24.400Z] Host lock lease acquired by instance ID '000000000000000000000000D7CD92BB'.
```

Step 4 — Upload a Test File to Blob Storage

14. Open a SECOND cmd.exe window (leave func start running in the first). Upload a test file:

```
az storage blob upload \
  --connection-string "$CONN_STRING" \
  --container-name uploads \
  --name testfile.txt \
  --data "hello from the lab"
```

```

dharshinik@DHC47TYX72-dharshinik BlobArchiveFunctions % az storage blob uplo
ad \
  --connection-string "$CONN_STRING" \
  --container-name "$CONTAINER_UPLOADS" \
  --name testFile.txt \
  --data "hello from the lab"
Argument '--data' is in preview and under development. Reference and support
levels: https://aka.ms/CLI_refstatus
Finished[#####] 100
.0000%
{
  "client_request_id": "f8b13a3a-75f5-11f1-94c7-5a3e3b7b403f",
  "content_md5": "xthBBgnWX/kbAfeF3PNJXQ==",
  "date": "2026-07-02T09:11:11+00:00",
  "encryption_key_sha256": null,
  "encryption_scope": null,
  "etag": "\"0x8DED819DCF8787B\"",
  "lastModified": "2026-07-02T09:11:12+00:00",
  "request_id": "0647b8b4-401e-0057-2702-0ae02f000000",
  "request_server_encrypted": true,
  "structured_body": null,
  "version": "2026-04-06",
  "version_id": null
}
dharshinik@DHC47TYX72-dharshinik BlobArchiveFunctions %

```

Home > dharshinifuncabsa29449 | Containers

dh

dharshini-uploads

Container

Search

+

Add Directory

↑

Upload

🔒

Change access level

🔄

Refresh

🗑️

Delete

📄

Copy

📄

Paste

🔄

Rename

🔑

Acquire lease

🔑

Break lease

📄

Edit columns

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

dh

dharshini-uploads

Authentication method: Access key (Switch to Microsoft Entra user account)

Add filter

Search blobs by prefix (case-sensitive)

Only show active blobs

Showing all 1 items

☒	Name	Last modified	Access tier	Blob type	Size	Lease state
☒	testFile.txt	2/7/2026, 2:41:12 PM	Hot (Inferred)	Block blob	18 B	Available

Tip

For testing the archive logic without waiting 7 days, you will use the `?days=0` query parameter in the next step — this archives ALL files regardless of age.

Step 5 — Trigger ManualArchive

15. In the second cmd.exe window, call the HTTP trigger with `days=0` to archive everything immediately:

```
curl -X POST "http://localhost:7071/api/ManualArchive?days=0"
```

Expected Result

Expected response: Archive complete. Files moved: 1

```

dharshinik@DHC47TYX72-dharshinik BlobArchiveFunctions % curl -X POST "http://localhost:7071/api/ManualArchive?days=0"
Archive complete. Files moved: 1%
dharshinik@DHC47TYX72-dharshinik BlobArchiveFunctions %

```

File Removed from Uploads:

Container: dharshini-uploads

Search:

Actions: + Add Directory, Upload, Change access level, Refresh, Delete, Copy, Paste, Rename, Acquire lease, Break lease, Edit columns

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

Authentication method: Access key ([Switch to Microsoft Entra user account](#))

Add filter

Search blobs by prefix (case-sensitive):

Only show active blobs:

Showing all 0 items

Name	Last modified	Access tier	Blob type	Size	Lease state
No items found					

File Moved to Archive:

Home > dharshinifunclabsa29449 | Containers

Container: dharshini-archive

Search:

Actions: + Add Directory, Upload, Change access level, Refresh, Delete, Copy, Paste, Rename, Acquire lease, Break lease, Edit columns

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

Authentication method: Access key ([Switch to Microsoft Entra user account](#))

Add filter

Search blobs by prefix (case-sensitive):

Only show active blobs:

Showing all 1 items

Name	Last modified	Access tier	Blob type	Size	Lease state
testFile.txt	2/7/2026, 2:46:57 PM	Hot (Inferred)	Block blob	18 B	Available

Step 6 — Verify the Blob Moved

16. Confirm testfile.txt is now in the archive container and gone from uploads:

```

az storage blob list --account-name $SA_NAME --container-name archive -o table
az storage blob list --account-name %SA_NAME% --container-name uploads --auth-mode login -o table

```

```

dharshinik@DHC47TYX72-dharshinik BlobArchiveFunctions % az storage blob list \
--account-name $SA_NAME \
--container-name dharshini-archive \
-o table

```

There are no credentials provided in your command and environment, we will query for account key for your storage account. It is recommended to provide `--connection-string`, `--account-key` or `--sas-token` in your command as credentials.

You also can add `'--auth-mode login'` in your command to use Azure Active Directory (Azure AD) for authorization if your login account is assigned required RBAC roles. For more information about RBAC roles in storage, visit <https://learn.microsoft.com/azure/storage/common/storage-auth-aad-rbac-cli>.

In addition, setting the corresponding environment variables can avoid inputting credentials in your command. Please use `--help` to get more information about environment variable usage.

Name	Blob Type	Blob Tier	Length	Content Type	Last Modified	Snapshot
testFile.txt	BlockBlob	Hot	18	application/octet-stream	2026-07-02T09:16:57+00:00	

testFile.txt is present in archive

```

dharshinik@DHC47TYX72-dharshinik BlobArchiveFunctions % az storage blob list \
--account-name $SA_NAME \
--container-name dharshini-uploads \
-o table

```

There are no credentials provided in your command and environment, we will query for account key for your storage account. It is recommended to provide `--connection-string`, `--account-key` or `--sas-token` in your command as credentials.

You also can add `'--auth-mode login'` in your command to use Azure Active Directory (Azure AD) for authorization if your login account is assigned required RBAC roles. For more information about RBAC roles in storage, visit <https://learn.microsoft.com/azure/storage/common/storage-auth-aad-rbac-cli>.

In addition, setting the corresponding environment variables can avoid inputting credentials in your command. Please use `--help` to get more information about environment variable usage.

testFile is removed from uploads

Lab 8: Deploy to Azure

Now deploy the Function App to Azure. The deployed version will read the connection string from Key Vault via a Key Vault Reference — a special app setting syntax that tells Azure to fetch the value automatically. Your function code stays exactly the same.

Step 1 — Create the Function App in Azure

17. Stop func start in the first window (Ctrl+C), then run:

```
az functionapp create \  
  --name $FUNC_NAME% \  
  --resource-group $RG \  
  --storage-account $SA_NAME \  
  --consumption-plan-location $LOCATION \  
  --runtime dotnet-isolated \  
  --functions-version 4 \  
  --os-type Windows
```

```

dharshinik@DHC47TYX72-dharshinik BlobArchiveFunctions % az functionapp creat
e \
> --name $FUNC_NAME \
> --resource-group $RG \
> --storage-account $SA_NAME \
> --consumption-plan-location $LOCATION \
> --runtime dotnet-isolated \
> --os-type Windows
No functions version specified so defaulting to 4.
Error while trying to create and configure an Application Insights for the F
unction App. Please use the Azure Portal to create and configure the Applica
tion Insights, if needed.
{
  "autoGeneratedDomainNameLabelScope": null,
  "availabilityState": "Normal",
  "clientAffinityEnabled": false,
  "clientAffinityPartitioningEnabled": null,
  "clientAffinityProxyEnabled": false,
  "clientCertEnabled": false,
  "clientCertExclusionPaths": null,
  "clientCertMode": "Required",
  "cloningInfo": null,
  "containerSize": 1536,
  "customDomainVerificationId": "46807E15E4B3288A1BCC91EA4477FF9CF6E3CE676FA
CDEA0663A4E096E2CCC01",
  "dailyMemoryTimeQuota": 0,
  "daprConfig": null,
  "defaultHostName": "dharshini-func-lab-app-29449.azurewebsites.net",
  "dnsConfiguration": {
    "dnsAltServer": null,
    "dnsLegacySortOrder": null,
    "dnsMaxCacheTimeout": null,
    "dnsRetryAttemptCount": null,
    "dnsRetryAttemptTimeout": null,
    "dnsServers": null
  }
}

```

Function App URL
(https://FUNC_NAME.azurewebsites.net)

http:// dharshini-func-lab-app-
29449.azurewebsites.net

Step 2 — Assign a Managed Identity

```

az functionapp identity assign \
--name $FUNC_NAME \
--resource-group $RG

```

18. Capture the identity's principal ID:

```
PRINCIPAL_ID=$(az functionapp identity show \
  --name $FUNC_NAME \
  --resource-group $RG \
  --query principalId \
  -o tsv)

echo $PRINCIPAL_ID
```

Command 1: Assigning Managed Identity

```
dhmarshinik@DHC47TYX72-dharshinik BlobArchiveFunctions % az functionapp identity assign \
  --name $FUNC_NAME \
  --resource-group $RG \
  {
    "principalId": "ebfc4adf-39ae-470d-9c1e-e8afd898bd86",
    "tenantId": "2a731c61-a2b2-4661-8409-5b861cf40d0c",
    "type": "SystemAssigned",
    "userAssignedIdentities": null
  }
```

Command 2: Assigning principal Id to an env

```
dhmarshinik@DHC47TYX72-dharshinik BlobArchiveFunctions % export PRINCIPAL_ID=
$(az functionapp identity show \
  --name $FUNC_NAME \
  --resource-group $RG \
  --query principalId \
  -o tsv)

dhmarshinik@DHC47TYX72-dharshinik BlobArchiveFunctions % echo $PRINCIPAL_ID
ebfc4adf-39ae-470d-9c1e-e8afd898bd86
```

Step 3 — Grant the Identity Key Vault Access

```
az keyvault set-policy \
  --name $KV_NAME \
  --object-id $PRINCIPAL_ID \
  --secret-permissions get list
```

```

dharshinik@DHC47TYX72-dharshinik BlobArchiveFunctions % az keyvault set-policy \
--name $KV_NAME \
--object-id $PRINCIPAL_ID \
--secret-permissions get list
{
  "id": "/subscriptions/e25db867-f63d-489a-8bce-7de21e5f94a1/resourceGroups/rg_dharshini/providers/Microsoft.KeyVault/vaults/dharshini-kv-29449",
  "location": "southindia",
  "name": "dharshini-kv-29449",
  "properties": {
    "accessPolicies": [
      {
        "applicationId": null,
        "objectId": "12900e06-4032-440a-8e35-46dd3958352a",
        "permissions": {
          "certificates": [
            "all"
          ],
          "keys": [
            "all"
          ],
          "secrets": [
            "all"
          ],
          "storage": [
            "all"
          ]
        },
        "tenantId": "2a731c61-a2b2-4661-8409-5b861cf40d0c"
      },
      {
        "applicationId": null,
        "objectId": "ebfc4adf-39ae-470d-9c1e-e8afd898bd86",
        "permissions": {
          "certificates": null,
          "keys": null,
          "secrets": [
            "list",
            "get"
          ]
        }
      }
    ]
  }
}

```

Step 4 — Set the Key Vault Reference App Setting

Instead of pasting the connection string directly, use a Key Vault Reference. Azure resolves the `@Microsoft.KeyVault(...)` syntax automatically and injects the real value at runtime:

```
az functionapp config appsettings set \
```

```
--name $FUNC_NAME \
--resource-group $RG \
--settings
"BlobStorageConnectionString=@Microsoft.KeyVault(VaultName=$KV_NAME;SecretName=BlobStorageCon

//Instead of directly storing the connection string in the function app, we are making the function to access the
string from key vault using "key vault reference "
```

Tip

This is Key Vault Reference syntax. Your function code reads BlobStorageConnectionString exactly as before, but the CLI quietly swaps in the real value from Key Vault before handing it to your code. No code changes required.

```
dharshinik@DHC47TYX72-dharshinik BlobArchiveFunctions % az functionapp config appsettings set \
--name $FUNC_NAME \
--resource-group $RG \
--settings "BlobStorageConnectionString=@Microsoft.KeyVault(VaultName=$KV_NAME;SecretName=BlobStorageConnectionString)"
[
  {
    "name": "FUNCTIONS_WORKER_RUNTIME",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "WEBSITE_USE_PLACEHOLDER_DOTNETISOLATED",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "FUNCTIONS_EXTENSION_VERSION",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "AzureWebJobsStorage",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "WEBSITE_CONTENTAZUREFILECONNECTIONSTRING",
    "slotSetting": false,
    "value": null
  },
  {
    "name": "WEBSITE_CONTENTSHARE",
```

Step 5 — Publish and Deploy

```
func azure functionapp publish $FUNC_NAME
```

```
Getting site publishing info...
[2026-07-02T09:59:04.634Z] Starting the function app deployment...
Creating archive for current directory...
Uploading 3.04 MB [#####]
Upload completed successfully.
Deployment completed successfully.
[2026-07-02T10:00:04.886Z] Syncing triggers...
Functions in dharshini-func-lab-app-29449:
  ManualArchive - [httpTrigger]
    Invoke url: https://dharshini-func-lab-app-29449.azurewebsites.net/api/ma
nualarchive
  NightlyArchive - [timerTrigger]
dharshinik@DHC47TYX72-dharshinik BlobArchiveFunctions %
```

Step 6 — Test the Deployed HTTP Trigger

19. Upload a test file to blob storage first:

```
az storage blob upload \
  --account-name $SA_NAME \
  --container-name uploads \
  --name cloudtest.txt \
  --data "uploaded for cloud test" \
```

20. Trigger the ManualArchive function in the cloud:

```
curl -X POST "https://$FUNC_NAME.azurewebsites.net/api/ManualArchive?days=0"
```

Expected Result

Expected response: Archive complete. Files moved: 1

```
Permanent variables usage:
Alive[#####] 100.0000
Finished[#####] 100.0000
%
{
  "client_request_id": "ff6c804e-75fc-11f1-9650-5a3e3b7b403f",
  "content_md5": "f0Wu4pfSFSUPfSJConFysw==",
  "date": "2026-07-02T10:01:30+00:00",
  "encryption_key_sha256": null,
  "encryption_scope": null,
  "etag": "\"0x8DED820E3AFE9BB\"",
  "lastModified": "2026-07-02T10:01:30+00:00",
  "request_id": "999711ee-f01e-00a0-3709-0ac6c4000000",
  "request_server_encrypted": true,
  "structured_body": null,
  "version": "2026-04-06",
  "version_id": null
}
```

File uploaded to “uploads container” in deployment

dhmarshini-uploads

Container

Search

+ Add Directory

Upload

Change access level

Refresh

Delete

Copy

Paste

Rename

Acquire lease

Break lease

Edit columns

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

dhmarshini-uploads

Authentication method: Access key (Switch to Microsoft Entra user account)

Add filter

Search blobs by prefix (case-sensitive)

Only show active blobs

Showing all 1 items

Name	Last modified	Access tier	Blob type	Size	Lease state
cloudtest.txt	2/7/2026, 3:31:30 PM	Hot (Inferred)	Block blob	23 B	Available

After Manually archiving it through API endpoint:

```
dhmarshinik@DHC47TYX72-dharshinik BlobArchiveFunctions % curl -X POST "https://$FUNK_NAME.azurewebsites.net/api/ManualArchive?days=0"
Archive complete. Files moved: 1%
```

Uploads:

dhmarshini-uploads

Container

Search

+ Add Directory

Upload

Change access level

Refresh

Delete

Copy

Paste

Rename

Acquire lease

Break lease

Edit columns

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

dhmarshini-uploads

Authentication method: Access key (Switch to Microsoft Entra user account)

Add filter

Search blobs by prefix (case-sensitive)

Only show active blobs

Showing all 0 items

Name	Last modified	Access tier	Blob type	Size	Lease state
No items found					

archive:
dharshini-archive

Container

Search

◊ ◀

+

Add Directory

↑

Upload

🔒

Change access level

🔄

Refresh

🗑️

Delete

📄

Copy

📄

Paste

🔄

Rename

🔗

Acquire lease

🔗

Break lease

🔗

Edit columns

📁 Overview

🔧 Diagnose and solve problems

🔑 Access Control (IAM)

⌵ Settings

📁 dharshini-archive

Authentication method: Access key (Switch to Microsoft Entra user account)

🔍

Add filter

🔍

Search blobs by prefix (case-sensitive)

Only show active blobs

Showing all 2 items

☐	Name	Last modified	Access tier	Blob type	Size	Lease state
☐	cloudtest.txt	2/7/2026, 3:33:10 PM	Hot (Inferred)	Block blob	23 B	Available
☐	testFile.txt	2/7/2026, 2:46:57 PM	Hot (Inferred)	Block blob	18 B	Available

21. Verify the blob moved to archive:

```
az storage blob list \
  --account-name $SA_NAME \
  --container-name archive \
  --auth-mode login \
  -o table
```

In addition, setting the corresponding environment variables can avoid inputting credentials in your command. Please use --help to get more information

Name	Blob Type	Blob Tier	Length	Content Type	Last Modified	Snapshot
cloudtest.txt	BlockBlob	Hot	23	application/octet-stream	2026-07-02T10:03:10+00:00	
testFile.txt	BlockBlob	Hot	18	application/octet-stream	2026-07-02T09:16:57+00:00	

dharshinik@DHC47DYX72-dharshinik-BlobArchiveFunctions %

Lab 9: Cleanup

Important

This permanently deletes all resources. Only run when you are completely finished with the lab.

```
az group delete --name %RG% --yes --no-wait
```

```

dharshinik@DHC47TYX72-dharshinik BlobArchiveFunctions % az group delete --name $RG --yes --no-wait
dharshinik@DHC47TYX72-dharshinik BlobArchiveFunctions % az group exists --name $RG
false
dharshinik@DHC47TYX72-dharshinik BlobArchiveFunctions %

```

Reflection Questions

1. What is the difference between a Timer Trigger and an HTTP Trigger? When would you choose one over the other?

* Timer Trigger runs automatically based on a schedule (using a cron expression). It does not require any user request.

* HTTP Trigger runs only when an HTTP request (GET, POST, etc.) is sent to the function's endpoint.

2. What does the cron expression 0 0 0 * * * mean? How would you change it to run every Monday at 6:00 AM?

means:

- Second = 0
- Minute = 0
- Hour = 0
- Every day of the month
- Every month
- Every day of the week

So it runs every day at 12:00 AM (midnight UTC).

3. What is a Key Vault Reference? How is it different from storing the connection string directly as an app setting?

A Key Vault Reference allows an Azure App Service or Azure Function to retrieve a secret (such as a connection string) directly from Azure Key Vault without storing the actual secret in the application's configuration.

4. The NightlyArchive function runs at midnight UTC. If your business is in India (IST = UTC+5:30), what cron expression would run it at midnight IST?

Azure Functions use UTC for Timer Triggers by default. Midnight IST (00:00) is 6:30 PM UTC on the previous day. Therefore, the cron expression is: 0 30 18 * * *

This runs the function every day at 18:30 UTC, which corresponds to 00:00 (midnight) IST on the following day.

Common Errors

Error	Fix
func: command not found	Azure Functions Core Tools not installed. Run: <code>npm install -g azure-functions-core-tools@4 --unsafe-perm true</code>
Variable not set (%SA_NAME% prints literally)	Session was closed. Re-run all set commands from Lab 1 using your saved SUFFIX value
BlobStorageConnectionString is not configured	Check local.settings.json — the value must be the actual connection string, not a placeholder
403 on Key Vault (deployed)	Managed Identity access policy not set or wrong principalId — re-run <code>az keyvault set-policy</code> with the correct PRINCIPAL_ID
Key Vault Reference shows as literal string	The Function App identity was not granted get permission on Key Vault secrets — check Step 3 of Lab 8
func start fails — port in use	Another process is using port 7071. Close it or run: <code>func start --port 7072</code>
dotnet build errors on the function files	Ensure the <code>func new</code> command ran successfully and both .cs files exist in the project folder
Blob not archived — count is 0	The file is newer than the days threshold. Test with <code>?days=0</code> to archive all files regardless of age